Experiment 2 - Working with Maven

1. Creating a Maven Project

- **Step 1: Open Terminal**
- **Step 2: Generate Project**

  If there is already a folder named MyMavenApp then change the -DrartifactId value to some other name or delete the MyMavenApp folder.

  ```
  mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp
  -DarchetypeArtifactId=maven-archetype-quickstart -
  DinteractiveMode=false
  ```

- **Step 3: Navigate to Project Directory**
- `cd MyMavenApp`

---

2. Maven Project Layout

To visualize the structure of the generated Maven project, you can use the `tree` command.

If `tree` is not already installed on your system, run the following command in your terminal:

```
sudo snap install tree
```

Then, navigate to your project directory (e.g., `cd MyMavenApp`) and run:

```
Tree
```

```
skit@MANJUSHREENAVEEN: ~    skit@MANJUSHREENAVEEN: ·
 fruits.txt          osprg2          prg1.c          try.c
 fruitscprg          osprg2.1        prg1os.c
 fruitscprg.c        osprg2.1.c      prgb1.c
skit@MANJUSHREENAVEEN:~$ cd MyMavenApp
skit@MANJUSHREENAVEEN:~/MyMavenApp$ tree
Command 'tree' not found, but can be installed with:
sudo snap install tree  # version 2.1.3+pkg-5852, or
sudo apt  install tree  # version 2.1.1-2ubuntu3.24.04.2
See 'snap info tree' for additional versions.
skit@MANJUSHREENAVEEN:~/MyMavenApp$ sudo snap install tree
[sudo] password for skit:
2026-01-28T14:37:07Z INFO Waiting for automatic snapd restart...
tree 2.1.3+pkg-5852 from 林博仁 Buo-ren Lin (brlin) installed
skit@MANJUSHREENAVEEN:~/MyMavenApp$ tree
.
├── pom.xml
└── src
    ├── main
    │   └── java
    │       └── com
    │           └── example
    │               └── App.java
    └── test
        └── java
            └── com
                └── example
                    └── AppTest.java

10 directories, 3 files
skit@MANJUSHREENAVEEN:~/MyMavenApp$
```

Key files and directories:

- `pom.xml`
- `src/main/java/com/example/App.java`
- `src/test/java/com/example/AppTest.java`

---

3. Understanding the POM File (`pom.xml`)

Basic Structure:

- **Step 1: Open the pom.xml file**

```
gedit pom.xml
```

- **Step 2: Remove the existing pom.xml file and paste the following**

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="<http://maven.apache.org/POM/4.0.0>"
         xmlns:xsi="<http://www.w3.org/2001/XMLSchema-instance>"
         xsi:schemaLocation="<http://maven.apache.org/POM/4.0.0>
<http://maven.apache.org/xsd/maven-4.0.0.xsd>">

  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>MyMavenApp</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <properties>
    <!-- Java version -->
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>

    <!-- JUnit and Surefire versions -->
    <junit.jupiter.version>5.10.1</junit.jupiter.version>
    <surefire.plugin.version>3.0.0-M7</surefire.plugin.version>
  </properties>

  <dependencies>
    <!-- JUnit 5 API for writing tests -->
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-api</artifactId>
      <version>${junit.jupiter.version}</version>
      <scope>test</scope>
    </dependency>

    <!-- JUnit 5 Engine for running tests -->
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-engine</artifactId>
      <version>${junit.jupiter.version}</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <!-- Compiler plugin for Java 17 -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.8.1</version>
        <configuration>
          <source>${maven.compiler.source}</source>
          <target>${maven.compiler.target}</target>
        </configuration>
      </plugin>

      <!-- Surefire plugin to run JUnit 5 tests -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
```

```
        <artifactId>maven-surefire-plugin</artifactId>
        <version>${surefire.plugin.version}</version>
        <configuration>
          <argLine>--add-opens java.base/java.lang=ALL-UNNAMED</argLine>
        </configuration>
      </plugin>
    </plugins>
  </build>

</project>
```

4. Updating the Default Java Files

The Maven archetype generated default main and test Java files. We will now replace their content with the following code.

- **Step 1: To open and edit the main application file (`App.java`), use the following command:**

```
gedit src/main/java/com/example/App.java
```

Replace the existing content with this code:

```
package com.example;

public class App {
    public String getMessage() {
        return "Hello World!";
    }

    public static void main(String[] args) {
        System.out.println(new App().getMessage());
    }
}
```

- **Step 2: To open and edit the test file (`AppTest.java`), use the following command:**

```
gedit src/test/java/com/example/AppTest.java
```

Replace the existing content with this code:

```
package com.example;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

class AppTest {
```

```
    @Test
    void testGetMessage() {
        App app = new App();
        assertEquals("Hello World!", app.getMessage(), "Message should be
'Hello World!'");
    }
}
```

---

5. Compiling the Project

Now that we've updated the source code, we need to compile the project. This will also run the tests defined in `src/test/java`. The `mvn clean install` command cleans the target directory, compiles the source code, runs the tests, and packages the project into a JAR file.

```
mvn clean install
```

You should see output indicating that the build was successful and the tests passed.

---

6. Running the Application

You can run the main class of your application directly using the Maven Exec Plugin.

```
java -cp target/classes com.example.App
```

- `java`: The command to execute a Java class file.
- `cp target/classes`: This sets the classpath. It tells the Java runtime where to look for compiled `.class` files. In a standard Maven project, the compiled main classes are placed in the `target/classes` directory.
- `com.example.App`: The fully qualified name of the class containing the `main` method you want to execute.

```
skit@MANJUSHREENAVEEN: ~    skit@MANJUSHREENAVEEN: ·
/1.0.8/plexus-1.0.8.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/
1.0.8/plexus-1.0.8.pom (7.2 kB at 51 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus
-container-default/1.0-alpha-8/plexus-container-default-1.0-alpha-8.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-
container-default/1.0-alpha-8/plexus-container-default-1.0-alpha-8.pom (7.3 kB at 41 kB/s
)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus
-utils/3.0.5/plexus-utils-3.0.5.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus
-digest/1.0/plexus-digest-1.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-
digest/1.0/plexus-digest-1.0.jar (12 kB at 55 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-
utils/3.0.5/plexus-utils-3.0.5.jar (230 kB at 655 kB/s)
[INFO] Installing /home/skit/MyMavenApp/target/MyMavenApp-1.0-SNAPSHOT.jar to /home/skit/
.m2/repository/com/example/MyMavenApp/1.0-SNAPSHOT/MyMavenApp-1.0-SNAPSHOT.jar
[INFO] Installing /home/skit/MyMavenApp/pom.xml to /home/skit/.m2/repository/com/example/
MyMavenApp/1.0-SNAPSHOT/MyMavenApp-1.0-SNAPSHOT.pom
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  59.751 s
[INFO] Finished at: 2026-01-28T14:52:28Z
[INFO] ------------------------------------------------------------------------
skit@MANJUSHREENAVEEN:~/MyMavenApp$ java -cp target/classes com.example.App
Hello World!
skit@MANJUSHREENAVEEN:~/MyMavenApp$ gedit src/main/java/com/example/App.java
skit@MANJUSHREENAVEEN:~/MyMavenApp$ java -cp target/classes com.example.App
```